INSTITUTE OF ENGINEERING AND MANAGEMENT

Course Name: **Advanced Programming (OOP) Lab**

Project Title: Mini Banking System using Layered Architecture

Student Name: Snehasmita Pal

Roll Number: 83

Instructor Name: *Dr. Swagatam Basu, Dr. Susovan Jana, Dr. Soumadip Biswas*

Spring 2026

2. PROBLEM STATEMENT

The objective of this project is to design and implement a Mini Banking System that simulates fundamental banking operations such as account creation, deposit, withdrawal, balance inquiry, and fund transfer between accounts. In real-world banking systems, it is essential to maintain accuracy, security, and modularity while handling financial transactions. This project aims to provide a simplified model of such a system while emphasizing clean software design principles.

The system is developed using a layered architecture consisting of presentation, service, and repository layers. This architectural approach ensures separation of concerns, where each layer is responsible for a specific functionality, thereby improving maintainability, scalability, and ease of testing. The presentation layer handles user interaction, the service layer manages business logic, and the repository layer is responsible for data storage and retrieval.

The project demonstrates key Object-Oriented Programming (OOP) concepts. Encapsulation is used to protect sensitive data such as account balance, inheritance is applied to create specialized account types like SavingsAccount and CheckingAccount, and polymorphism is utilized to allow different behaviors for similar operations such as withdrawal. Abstraction is achieved through the service layer, which hides complex internal logic from the user.

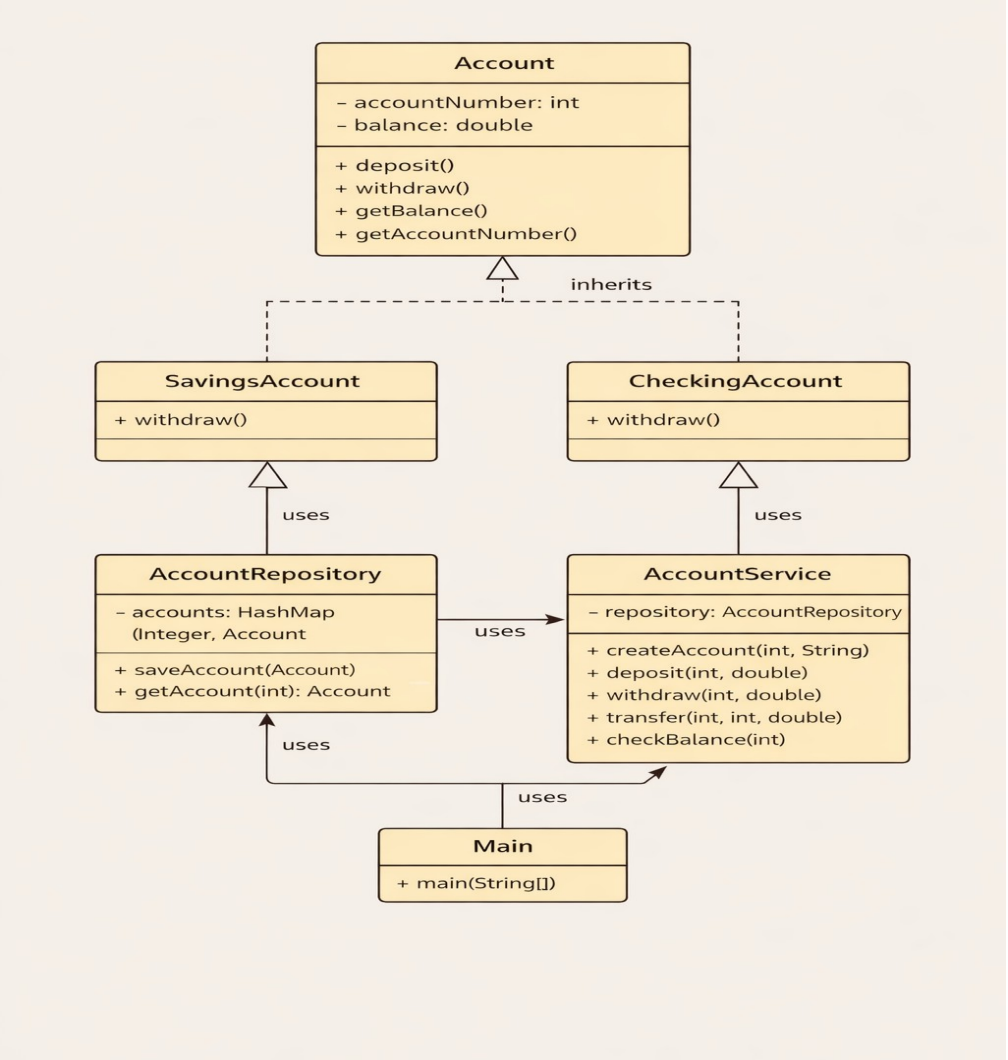
Additionally, the system incorporates input validation and error handling mechanisms to ensure robustness. Scenarios such as invalid account numbers, insufficient balance, and incorrect transaction amounts are properly handled to prevent system failure.

Overall, the project serves as a practical implementation of OOP principles and layered architecture in developing a structured and maintainable software system.

3. SYSTEM DESIGN

* UML Class Diagram

The UML class diagram represents the overall structure of the system, showing the relationships



between different classes such as Account, SavingsAccount, CheckingAccount, AccountService, AccountRepository, and Main. It highlights inheritance, association, and dependency relationships used in the system.

* Description of major classes

Account (Base Class)

The Account class is the core entity of the system. It contains common attributes such as accountNumber and balance. It provides basic functionalities like deposit, withdraw, and balance inquiry. This class serves as a parent class for different account types.

SavingsAccount

SavingsAccount extends the Account class and overrides the withdraw method to enforce standard withdrawal rules such as restricting withdrawal when balance is insufficient. It demonstrates inheritance and method overriding.

CheckingAccount

CheckingAccount also extends the Account class but allows overdraft up to a certain limit. It overrides the withdraw method to implement this behavior, thereby demonstrating polymorphism.

AccountRepository

The AccountRepository class is responsible for storing and retrieving account data. It uses a HashMap to store account objects using account numbers as keys. This layer abstracts data storage operations from business logic.

AccountService

The AccountService class contains the core business logic of the system. It acts as an intermediary between the UI and the repository. It handles operations such as account creation, deposit, withdrawal, transfer, and balance checking while also performing validation.

Main (UI Layer)

The Main class represents the presentation layer of the system. It provides a console-based interface that allows users to interact with the system through a menu-driven approach.

* <u>Package structure</u>

The project is organized into multiple packages to maintain modularity and separation of concerns:

model/ → Contains entity classes (Account, SavingsAccount, CheckingAccount)
service/ → Contains business logic (AccountService)
repository/ → Handles data storage (AccountRepository)
ui/ → User interface (Main class)

This structure improves code readability, maintainability, and scalability.

<u>* Interaction Flow</u>

The system follows a layered interaction model:

1. The user interacts with the system through the UI (Main class).
2. The UI sends requests to the Service layer.
3. The Service layer processes the request and applies business logic.
4. The Repository layer stores or retrieves data.
5. The result is returned back to the user through the UI.

> Flow Representation:

User → UI → Service → Repository → Response

4. OOP Concepts Demonstrated

◇ Encapsulation

Where used: Account class
Why: Protect balance data

```
private double balance;
public double getBalance() {
   return balance;
}
```

Explanation:
Data is hidden and accessed through methods ensuring security.

◇ Inheritance

Where used: SavingsAccount, CheckingAccount

```
class SavingsAccount extends Account {
}
```

Explanation:
Derived classes reuse base class properties.

◇ Polymorphism

Where used: withdraw() method

```
public void withdraw(double amount) {
   if(amount > balance) {
      System.out.println("Insufficient funds");
   }
}
```

Explanation:
Same method behaves differently for different classes.

◇ Abstraction

Where used: Service layer

```
public void transfer(int from, int to, double amount) {
   // hidden logic
}
```

Explanation:
User does not see internal implementation.

5. Implementation Highlights

Code snippet.1: Use of HashMap for Data Storage

```
Map<Integer, Account> accounts = new HashMap<>();
```

Justification:
A HashMap is used in the repository layer to store account objects with the account number as the key. This allows constant-time (O(1)) access for retrieving and updating account information, making the system efficient for operations like deposit, withdrawal, and balance checking. It also simplifies data management without requiring a database.

Code snippet.2 : Account Creation Logic

```
if (type.equalsIgnoreCase("savings")) {
   acc = new SavingsAccount(accNo);
} else {
   acc = new CheckingAccount(accNo);
}
```

Justification:
This logic dynamically creates objects based on user input, enabling support for multiple account types. It improves flexibility and demonstrates the use of polymorphism and object-oriented design principles.

Code snippet.3 Transfer Operation Implementation

```
Account sender = repo.getAccount(from);

Account receiver = repo.getAccount(to);

if (sender == null || receiver == null) {

   System.out.println("Invalid accounts");

   return;

}
```

Justification:
This snippet ensures that both sender and receiver accounts exist before performing a transaction. It prevents invalid operations and enhances system reliability by handling potential runtime errors.

Code snippet.4 Input Validation Mechanism

```
if (amount <= 0) {
System.out.println("Invalid amount");
return;
}
```

Justification:
Input validation ensures that only meaningful and valid data is processed by the system. This prevents logical errors such as negative transactions and improves overall robustness.

6. Testing & Error Handling

◇ Test Cases Considered

The following test cases were executed to validate the functionality of the system:

- **Account Creation:**
  Verified that new accounts (Savings/Checking) are created successfully with valid account numbers.
- **Deposit Operation:**
  Tested depositing valid amounts and ensuring balance updates correctly.
- **Withdrawal Operation:**
  Checked withdrawal within balance limits and verified balance deduction.
- **Fund Transfer:**
  Tested transferring money between two valid accounts.
- **Balance Inquiry:**
  Verified that correct balance is displayed for a given account.

◇ Edge Cases Handled

The system also handles various edge cases to prevent unexpected behavior:

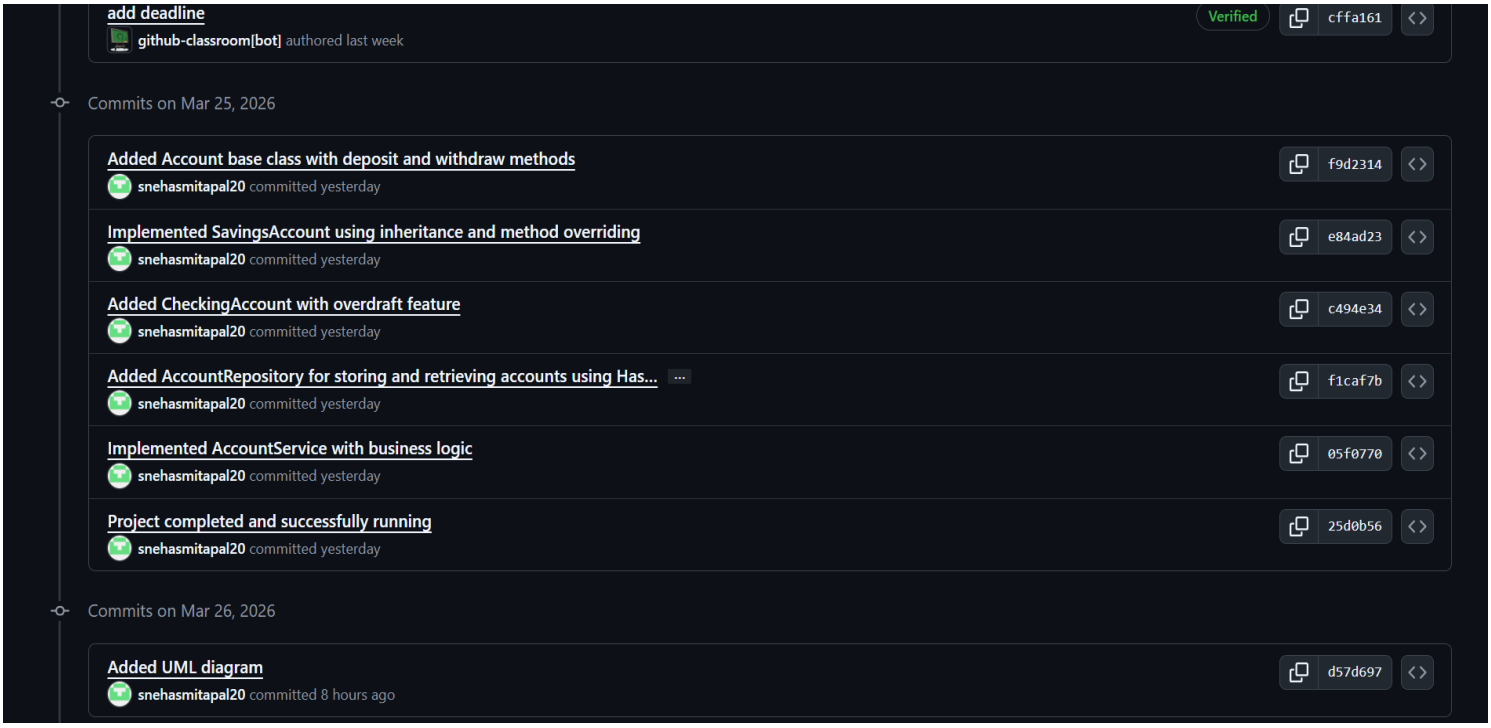
- **Negative or Zero Amount:**
  The system rejects deposits or transfers with non-positive values.
- **Excess Withdrawal:**
  Withdrawal requests exceeding the available balance are restricted.
- **Invalid Account Number:**
  Operations on non-existing accounts are prevented.
- **Same Account Transfer:**
  The system avoids transferring money to the same account.

◇ Failure Scenarios

The following failure scenarios are properly handled with appropriate messages:

- **Account Not Found:**
  Displays an error when an invalid account number is entered.
- **Insufficient Balance:**
  Prevents withdrawal when funds are insufficient.
- **Invalid Transaction Amount:**
  Rejects negative or zero transaction amounts.
- **Overdraft Limit Exceeded (Checking Account):**
  Prevents withdrawal beyond allowed overdraft limit.

7. Git Workflow



Explanation:

The project was developed incrementally using multiple commits across several days. Each commit represents a specific feature such as model creation, repository implementation, service logic, and UI integration. This demonstrates proper version control practices.

8. Conclusion & Future Scope

Conclusion:

The project successfully demonstrates a modular banking system using layered architecture and OOP concepts. It ensures maintainability and structured design.

Future Scope:

- GUI integration
- Database support
- User authentication

9.Appendix

Initial project proposal :

Name: Snehasmita Pal

Roll No.:83

Dept.:IOT

Project Title
Mini Banking System using Layered Architecture (Service + Repository)

Problem Statement :
Manual banking operations are often slow and prone to errors when managing customer accounts and financial transactions. A simple software-based system can help organize banking activities more efficiently. The Mini Banking System is designed to simulate basic banking operations such as account creation, deposits, withdrawals, and balance inquiries using Java and object-oriented programming principles. The project uses a layered architecture to separate the application into different layers, including presentation, service, and repository. This structure improves code organization and maintainability. The system will demonstrate how OOP concepts can be used to build a modular and structured application that manages banking operations effectively.

Target User:

- Bank staff handling customer account operations
- Students learning banking system concepts

Core Features:

- Create a new bank account
- Deposit and withdraw money
- Check account balance
- View account details
- Basic input validation

OOP Concepts to be Used:

Abstraction – Interfaces or abstract classes define common banking operations.

Inheritance – Account types like SavingsAccount inherit from a base Account class.

Polymorphism – Methods such as deposit() and withdraw() behave differently for different account types.

Exception Handling – Handles invalid inputs and insufficient balance errors.

Collections – Java collections (ArrayList/HashMap) store and manage multiple accounts.

Proposed Architecture Description:
The system follows a layered architecture with three layers: presentation, service, and repository. The presentation layer manages user interaction, the service layer processes banking operations and business logic, and the repository layer handles data storage using Java collections. This separation improves code clarity and maintainability.

Copy of the review mail receied (with APPROVED_MINOR):

PCCCS495 Proposal Review – APPROVED_MINOR

S

soumadip.biswas@ieminternational.ai

To: Snehasmita Pal

Wed 3/11/2026 9:06 AM

Dear Snehasmita Pal ,

Project: Mini Banking System using Layered Architecture (Service + Repository)
Decision: APPROVED_MINOR
Score: 90

Strengths:
The proposal presents a clear and well-defined problem statement, along with a suitable scope for a university project. The choice of a layered architecture (presentation, service, repository) is excellent for demonstrating separation of concerns, modularity, and maintainability, which are key learning objectives for OOP and software design. The explicit mention of core OOP concepts (Abstraction, Inheritance, Polymorphism, Exception Handling, Collections) ensures a strong focus on the course material. The features are appropriate and cover basic banking operations effectively. Using Java Collections for data storage is a pragmatic choice for a 'mini' system, allowing students to concentrate on architectural patterns and OOP principles without the added complexity of database integration.

Improvements:
To further strengthen the proposal, consider briefly elaborating on *how* specific